

Para esta atividade você pode usar o banco de dados da FIAL ou o banco que está na máquina virtual usada nas aulas

1) Crie uma tabela MY_CALENDAR particionada que armazena informações de calendário. A tabela deve ser particionada usando INTERVALO de um mês.

➔ Note que a opção INTERVAL automatiza a criação de novas partições.

```
CREATE TABLE MY_CALENDAR (  
  id number GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  calendar_date DATE not null,  
  day_of_week VARCHAR2(10),  
  day_of_month VARCHAR2(2),  
  month VARCHAR2(2),  
  year NUMBER(4),  
  quarter VARCHAR2(1),  
  is_weekend VARCHAR2(1)  
)  
PARTITION BY RANGE (calendar_date)  
INTERVAL (NUMTOYMINTERVAL(1, 'MONTH')) -- por mês  
(  
  PARTITION p_2000_q1 VALUES LESS THAN (TO_DATE('01-APR-2000', 'DD-MON-YYYY')),  
  PARTITION p_2000_q2 VALUES LESS THAN (TO_DATE('01-JUL-2000', 'DD-MON-YYYY')),  
  PARTITION p_2000_q3 VALUES LESS THAN (TO_DATE('01-OCT-2000', 'DD-MON-YYYY')),  
  PARTITION p_2000_q4 VALUES LESS THAN (TO_DATE('01-JAN-2001', 'DD-MON-YYYY'))  
);
```

2) Carregue dados para 30 anos iniciando do ano 2000. Note que você pode especificar partições iniciais

```
INSERT INTO MY_CALENDAR  
(calendar_date, day_of_week, day_of_month, month, year, quarter, is_weekend)  
SELECT  
  TRUNC(DATE '2000-01-01') + (LEVEL - 1) AS calendar_date,  
  TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'D') AS day_of_week,  
  TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'DD') AS day_of_month,  
  TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'MM') AS month,  
  TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'YYYY') AS year,  
  TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'Q') AS quarter,  
  CASE WHEN TO_CHAR(TRUNC(DATE '2000-01-01') + (LEVEL - 1), 'D') IN ('1', '7') THEN 'Y' ELSE 'N' END  
  AS is_weekend  
FROM  
  dual  
CONNECT BY  
  TRUNC(DATE '2000-01-01') + (LEVEL - 1) <= ADD_MONTHS(DATE '2000-01-01', 12 * 30) - 1;  
  
commit;
```

3. Vamos consultar algumas informações do dicionário de dados referente as tabelas particionadas criadas pelo usuário:

Execute os seguintes comandos SELECT e analise o resultado de cada um:

```
select table_name, partitioning_type, partition_count, interval from user_part_tables;

select column_name from USER_PART_KEY_COLUMNS where name = 'MY_CALENDAR';

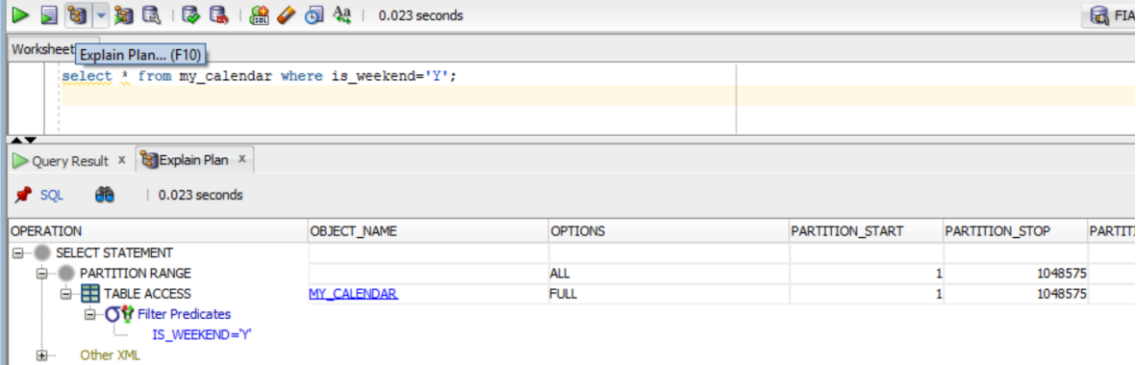
select table_name, partition_position, partition_name, high_value from user_tab_partitions;
```

4. Agora vamos consultar datas nessa tabela e analisar como o SGBD se comporta ao executar a consulta:

a) Mostrar todos as datas que são finais de semana:

```
select * from my_calendar where is_weekend='Y';
```

b) Use o botão “Plano de Explicação” para mostrar o plano de execução da consulta



OPERATION	OBJECT_NAME	OPTIONS	PARTITION_START	PARTITION_STOP	PARTITION_POSITION
SELECT STATEMENT					
PARTITION RANGE		ALL		1048575	
TABLE ACCESS	MY_CALENDAR	FULL		1048575	
Filter Predicates					
IS_WEEKEND='Y'					

- Quantas partições foram lidas?
- Qual o custo da operação?

c) Agora vamos modificar a consulta para incluir a coluna que é chave de partição da tabela (calendar_date).

-- retornar as datas que são finais de semana entre Janeiro e Junho de 2025.

```
select * from my_calendar where is_weekend='Y'
and calendar_date between to_date( '2025-01-01', 'YYYY-MM-DD') and to_date( '2025-06-30', 'YYYY-MM-DD')
order by calendar_date;
```

- Quantas partições foram lidas?
- Qual o custo da operação?

d) Agora consulte todas as informações de uma data em específico

```
select * from my_calendar where calendar_date = to_date( '2010-05-01', 'YYYY-MM-DD');
```

- Quantas partições foram lidas?
- Qual o custo da operação?

Desafios

1) Agora vamos supor que você quisesse que a tabela MY_CALENDAR tivesse partições por ano.

1. Remova a tabela MY_CALENDAR (drop table my_calendar)
2. Recrie a tabela MY_CALENDAR de forma que tenha uma partição para cada ano:
Como referência, a consulta abaixo retorna a data de hoje e a data daqui a 1 ano

```
SELECT SYSDATE, SYSDATE + (INTERVAL '1' YEAR) FROM DUAL;
```

3. Execute as mesmas consultas dos exercícios anteriores, mas agora considere filtrar por ano em vez da data.

2) Agora vamos supor que você quisesse que a tabela MY_CALENDAR tivesse apenas 12 partições, uma para cada mês do ano.

4. Remova a tabela MY_CALENDAR (drop table my_calendar)
5. Recrie a tabela MY_CALENDAR de forma que tenha uma partição para cada mês
6. Execute as mesmas consultas dos exercícios anteriores, mas agora considere filtrar por mês em vez da data